

Google Play Store readiness

Budger | Read-only technical audit

CURRENT STATUS: NOT GOOGLE PLAY-READY YET

The repository contains an Android section inside an Expo-style configuration, but it does not contain a buildable Android project or signed Android App Bundle.

What is already in good shape

- Production frontend is reachable at <https://budger.app>
- Production backend health endpoint responds successfully
- The production Vite frontend build passes
- Privacy policy includes data export, account deletion, receipt/photo handling, and opt-in crash diagnostics
- In-app data export endpoint exists
- Account deletion request flow exists
- Crash reporting is opt-in by default
- Receipt and screenshot imports use browser file and camera capabilities
- The web app has notification permission and Web Push subscription logic
- The existing legal text names Google LLC as an Android platform service provider

Current Android/native project state

The repository search found no Android project files. There is no AndroidManifest.xml, Gradle project, Gradle wrapper, package.json for a native app, google-services.json, EAS configuration, or Capacitor configuration.

The only Android-specific configuration is an android section inside `export/ios/app.config.ts`. That file is explicitly described as an iOS-focused Expo configuration and references assets that are not present.

The current configuration references:

```
export/ios/assets/adaptive-icon.png
```

That asset is missing. There are also no Android launcher icon sets, splash assets, adaptive icon XML files, or native resource directories.

The configuration contains a placeholder EAS project identifier:

```
REPLACE_WITH_EAS_PROJECT_ID
```

This means the current Android material is a preparation template, not a project that can be opened in Android Studio or uploaded to Google Play Console.

What is required before a Play upload

- A real native Android project, or a configured Expo/EAS or Capacitor project
- A unique application ID/package name and final app name
- Android launcher icons, adaptive icons, splash assets, and store graphics
- A release build targeting the current Google Play API requirement

- A signed Android App Bundle (.aab), not only a web build or unsigned APK
- Play App Signing enrollment and an upload key
- A reproducible release build process
- Testing on supported Android versions, screen sizes, and network conditions
- A Google Play Console app entry and completed store listing
- Content rating questionnaire, target audience declaration, and app-access instructions
- Data Safety form and privacy-policy URL
- App screenshots, short description, full description, category, and contact details

Important technical concern: the product is currently web-first

The application currently relies on React/Vite, browser localStorage, browser service workers, browser Notification, browser PushManager, VAPID Web Push, and browser camera/file APIs.

A basic WebView wrapper would not automatically provide:

- Firebase Cloud Messaging or reliable native Android push-token registration
- Android notification channels and notification runtime permission handling
- Native background work and boot-time scheduling
- Native camera and photo-picker integrations
- Native biometric authentication
- Reliable Web Push behavior inside every WebView configuration
- Native offline behavior and lifecycle handling
- Play-compliant native value beyond simply displaying the website

The current frontend registers a browser service worker at /sw.js and uses PushManager with a VAPID key. That is a web-push design, not an Android FCM design. A native Android wrapper would need an intentional push architecture and backend token registration path.

Google Play may question or reject a thin website wrapper if the submitted app does not provide meaningful native-app functionality. The final approach should therefore be chosen deliberately: native Android/Expo features, a carefully configured hybrid shell, or a well-tested trusted-web-activity/PWA strategy where appropriate.

Android permissions and declarations needing verification

The current Android section declares:

- android.permission.CAMERA
- android.permission.READ_EXTERNAL_STORAGE
- android.permission.RECEIVE_BOOT_COMPLETED
- android.permission.VIBRATE

These declarations are not backed by a compiled native project yet. Before submission:

- Use the modern Android photo picker or scoped media permissions where possible; READ_EXTERNAL_STORAGE is obsolete or ineffective on newer Android versions.
- Add POST_NOTIFICATIONS for Android 13+ if the native app posts notifications, and request it at an appropriate user-driven moment.
- Keep CAMERA only if the native app actually invokes native camera functionality.
- Keep RECEIVE_BOOT_COMPLETED only if real native scheduled work needs to resume after reboot.
- Do not declare permissions merely for future features.

- Verify all merged permissions in the final AAB and document them in Play Console if required.

Google Play target API and packaging

Google's current target-SDK guidance says that, starting August 31, 2026, new apps and app updates must target Android 16 (API level 36) or higher, with limited platform-specific exceptions. For a release planned before that date, the applicable current Play requirement must be checked again at upload time.

Google Play uses the Android App Bundle format for distribution. The deliverable must therefore be a signed .aab produced from the native Android build, with Play App Signing configured. The current repository contains neither an Android build nor an AAB.

The final release should also be checked for 64-bit support, modern SDK compatibility, dependency security, network security, backup behavior, deep links, and correct handling of Android process death and rotation.

Data Safety and privacy status

The privacy foundation is promising but the Play Console declarations have not been completed or verified against a native dependency tree.

Likely data categories requiring review include:

- Name and email address, linked to the user's account
- Financial information, including transactions, budgets, goals, and recurring payments
- Photos or videos for receipts and bank/screenshot imports
- Device or push-token identifiers when notifications are enabled
- Crash and performance diagnostics when the user opts in
- Authentication/session data and account-deletion requests

Before submission, the owner must confirm:

- Whether each data type is collected, shared, optional, encrypted in transit, and linked to the user
- The purpose for each category and the retention period
- All third-party processors, including Vercel, Render, Neon, Supabase, Sentry, and Google services
- Whether crash data can include screen replay or financial information visible on screen
- Whether the privacy policy accurately describes Android-specific behavior
- That the Data Safety form matches the final native SDKs and their automatic collection behavior

Account deletion and user access

The app already has an account-deletion request flow and a data-export endpoint, which is a strong starting point.

For Google Play, verify the complete submitted-app flow:

- A user can initiate deletion from inside the app.
- The deletion request actually removes the account and associated data according to the stated policy.
- The Play Console account-deletion declaration is completed.

- If a web URL is required for deletion requests, it is publicly reachable, clearly branded, and does not require unnecessary sign-in.
- The export action returns the user's data in a usable format.
- Receipt images and third-party storage records are included in deletion handling.

Monetization and policy checks

No Google Play Billing implementation was found in the audited source. If Budger later sells digital subscriptions, premium features, or other in-app digital goods, Google Play Billing and the relevant payments policy must be evaluated before release.

If the app remains free with no ads and no digital purchases, the store listing and Data Safety answers should say so accurately. Do not add billing, advertising, or analytics declarations unless those SDKs and flows are actually present.

Testing and release checklist

- Build and install a release-signed AAB on physical Android devices.
- Test login, session persistence, CSRF-protected mutations, export, and account deletion.
- Test receipt camera capture, image-picker import, compression, upload, and deletion.
- Test notification permission, notification delivery, deep links, and disabled-notification behavior.
- Test offline/online transitions, service-worker behavior, and native lifecycle events.
- Test Android 13+ notification permission and modern photo-picker behavior.
- Test dark mode, small screens, tablets if supported, keyboard behavior, and rotation policy.
- Run a release build dependency/security audit.
- Use Play internal testing before production rollout.
- Prepare a reviewer account and clear app-access instructions.

Overall assessment

- Web/PWA production readiness: Good
- Privacy-feature readiness: Mostly good
- Android native-wrapper readiness: Early scaffold only
- Google Play upload readiness: Not ready
- Native push readiness: Not ready; current implementation is Web Push
- Android release artifact: Missing; no signed AAB
- Estimated remaining work: A proper native or hybrid Android implementation, not merely packaging

Once the native approach is selected - Expo/React Native, Capacitor, or another intentional Android strategy - the project can be prepared with native permissions, push registration, Android assets, a reproducible signed AAB build, Play Console metadata, and release testing.

Official Google reference basis

- Target API requirements:
<https://developer.android.com/google/play/requirements/target-sdk>
- Data Safety section:
<https://support.google.com/googleplay/android-developer/answer/10787469>

- Account deletion requirements:
<https://support.google.com/googleplay/android-developer/answer/13327111>
- Android App Bundle format:
<https://developer.android.com/guide/app-bundle/app-bundle-format>
- Upload a bundle to Play: <https://developer.android.com/studio/publish/upload-bundle>
- Google Play policies: <https://developer.android.com/distribute/play-policies>